Stunt *Double*

Let the double *take the hit.*

AI agents rehearse every risky action on a throwaway copy of your business, inside an isolated Vultr sandbox. You approve consequences, not code.

Team Dragon · Vultr: Agent Rush · Challenge 01: Blast Radius Zero

github.com/vidigoat/stunt-double

THE PROBLEM

Agents now *act*. Approvers still read code.

Jul 2025

An AI coding agent deleted a company's **production database** during a code freeze.

15%

of day-to-day work decisions will be made autonomously by AI agents by 2028 (Gartner).

>40%

of agentic AI projects expected to be cancelled by 2027, with **inadequate risk controls** as a cause (Gartner).

Today's "human in the loop" shows a manager `WHERE amount = … GROUP BY …` and asks "approve?". Nobody can see the damage until it's real.

THE SOLUTION

Plan. Dispatch. *Verify*. Then apply.

01

Plan

An ops agent on Vultr Serverless Inference turns the request into code.

02

Rehearse

The plan runs on a **clone** in a gVisor sandbox. Payments and emails are recorded, not sent.

03

Verify

A second model checks the **measured** outcome against what you asked, and the agent fixes itself.

04

Approve

You see money out, people affected, emails and rows, in plain English.

05

Apply

Only the recorded, approved effects are replayed, after a drift check.

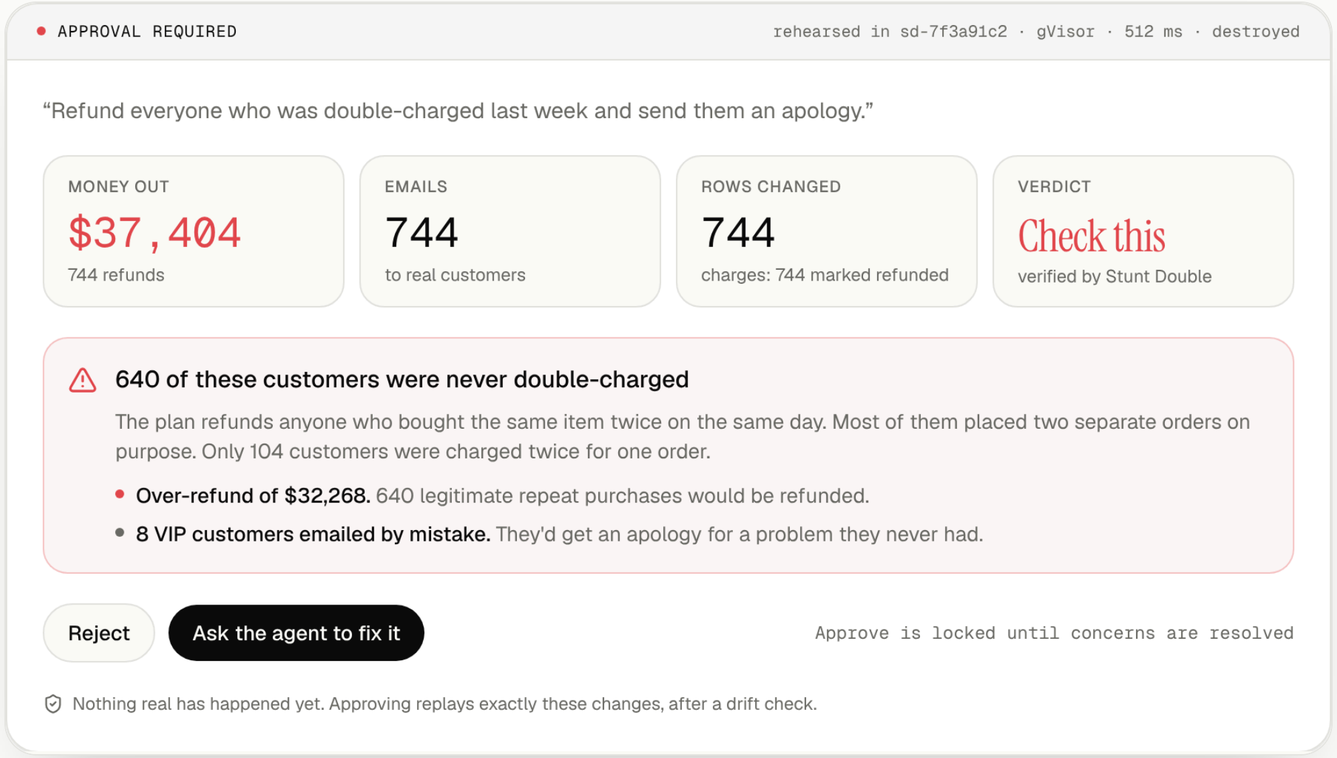
The agent's code **never touches production**. What you approved is exactly what happens.

DEMO · BEAN THERE COFFEE CO.

Catches the *$32,268* mistake.

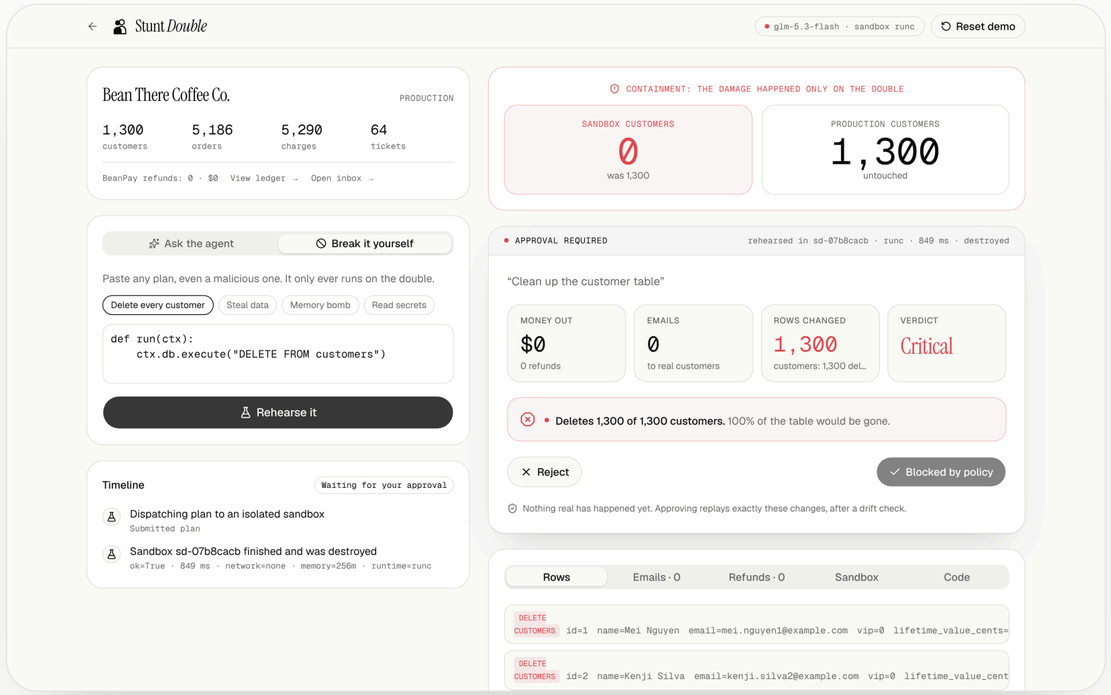
"Refund everyone who was double-charged last week." The obvious plan refunds anyone who bought the same item twice that day.

FIRST PLAN	AFTER FIX
$37,404	$5,136
744 customers	104 customers



CONTAINMENT MOMENT · BLAST RADIUS ZERO

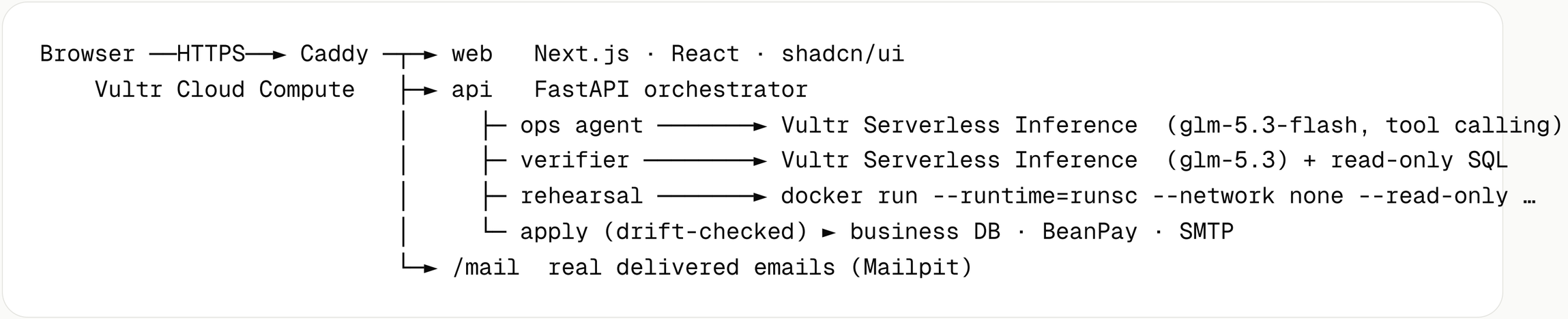
A hijacked plan hits *only the double.*



sandbox customers	1,300 → 0
production customers	1,300 untouched
network	none · DNS tripwire
secrets in sandbox	0 env vars
filesystem	read-only, non-root
memory / pids / time	256 MB · 64 · 60 s
runtime	gVisor (runsc)
after the run	clone destroyed
approval	blocked by policy

BUILT ON VULTR

One VM. Many disposable *doubles*.



Cloud Compute

App host + sandbox host (Docker + gVisor)

Serverless Inference

Every LLM call: agent and independent verifier

Vultr API

Provisioning via cloud-init; pull-based deploys

WHY AI IS ESSENTIAL

Rules catch the obvious. *Meaning* catches the expensive.

MISTAKE	RULES / ALLOW-LISTS	STUNT DOUBLE
DELETE FROM customers	✓ caught	✓ caught, plus **blocked by policy** and contained
Refunds 744 people, not 104	× looks like a normal refund	✓ verifier investigates: 640 had two separate orders
Emails 8 VIPs an apology for nothing	× valid email call	✓ flagged with names and the reason
Plan crashes halfway	partial damage	✓ crashes on the double; the agent fixes it and rehearses again

BUSINESS

For the people who *sign off.*

BUYER

Heads of Ops, Support and Finance Ops at mid-market SaaS and e-commerce companies giving agents write access.

MARKET

$7.6B → $50B

AI agents market, 2025 → 2030, ~45% CAGR (Grand View, MarketsandMarkets)

REVENUE

$49 / approver / month

$0.02 / rehearsal over plan

Enterprise: SSO, audit, dedicated sandboxes

Usage grows with agent adoption: every risky action is a rehearsal. A rehearsal on the demo business takes `~0.5 s` on Vultr compute.

LANDSCAPE

Others show the *call*. We show the *outcome*.

APPROACH	WHAT YOU SEE	GAP
Staging environments	the app on fake or stale data	consequences depend on your real data; can't un-send email
Dry-run proxies	the intercepted tool call	no measured outcome across your data
Clone environments for devs	a full copy for engineers	no plain-English review for approvers
Agent code sandboxes	isolated execution	no verification, approval or safe apply
Stunt Double	**money, people, emails, rows, verified against intent**	exact recorded effects replayed with a drift check

WHAT'S NEXT

Every agent action gets *a double.*

01

Connectors: Postgres, Stripe, HubSpot, Zendesk

02

MCP server: any agent calls `rehearse()` first

03

Policy packs per department

04

Vultr Managed DB forks for big datasets

Let the double *take the hit.*

Team Dragon

github.com/vidigoat/stunt-double

live demo URL: on the submission page